AgentBridge AI Studio

Local Setup and Run Guide

Python 3.12 | FastAPI | SQLite | Ollama LLM Runtime | WhatsApp Cloud API

Purpose
This guide explains how to install Ollama, configure the local environment, seed the workspace, run the FastAPI server, validate the LLM connection, execute workflows, and troubleshoot the common Windows issues encountered during setup.

Setup Overview

The platform runs fully on a local machine. Ollama provides the local LLM runtime. FastAPI serves the backend and web UI. SQLite stores agents, workflows, messages, logs, memory, and metrics.

Component	Purpose	Required Value / Command
Python	Application runtime and package manager	Python 3.12 recommended
Ollama	Local LLM server for agent responses	llama3.2:3b
FastAPI	Backend API and web UI server	http://127.0.0.1:8000
SQLite	Local persistence layer	data/agentbridge.db
WhatsApp	External messaging channel	Meta Cloud API credentials

1. Install Ollama on Windows

Open PowerShell and run the official Windows install command:

```
irm https://ollama.com/install.ps1 | iex
```

After installation, close VS Code completely and reopen it so the terminal can refresh PATH.

```
ollama --version
```

If ollama is not recognized
Locate the executable and add it to PATH using the commands in Step 2. This is common on Windows when VS Code was already open before Ollama was installed.

2. Fix Ollama PATH if Needed

If PowerShell says ollama is not recognized, locate the executable:

```
Get-ChildItem -Path "$env:LOCALAPPDATA\Programs", "$env:ProgramFiles", "$env:ProgramFiles(x86)" -Filter "ollama.exe"
-Recurse -ErrorAction SilentlyContinue | Select-Object FullName
```

Expected path:

```
C:\Users\<YourUser>\AppData\Local\Programs\Ollama\ollama.exe
```

Add it to PATH for the current terminal:

```
$env:Path += ";$env:LOCALAPPDATA\Programs\Ollama"
ollama --version
```

Make the PATH update permanent:

```
$ollamaPath = "$env:LOCALAPPDATA\Programs\Ollama"
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")

if ($userPath -notlike "*$ollamaPath*") {
    [Environment]::SetEnvironmentVariable("Path", "$userPath;$ollamaPath", "User")
}
```

3. Download and Test the Local LLM Model

Download the model used by the agents:

```
ollama pull llama3.2:3b
```

Verify the model is available:

```
ollama list
```

Expected output should include llama3.2:3b. Then test the local Ollama API:

```
Invoke-RestMethod `
  -Uri "http://127.0.0.1:11434/api/generate" `
  -Method POST `
  -ContentType "application/json" `
  -Body '{
    "model": "llama3.2:3b",
    "prompt": "Say hello in one short sentence.",
    "stream": false
  }'
```

If the response field says Hello or similar, the LLM runtime is working.

4. Open the Project Folder

Open VS Code and open the project root folder. The root folder should contain app/, scripts/, tests/, README.md, and requirements.txt.

```
cd "C:\Users\MdAzharuddinAnsari\Downloads\agentbridge_ai_studio_challenge_exact_fastfix\agentbridge_ai_studio"
```

Folder name may differ

If your extracted folder has a different name, use that folder path instead. Do not cd into agentbridge_ai_studio again if you are already inside it.

5. Create and Activate Python Virtual Environment

Create the virtual environment using Python 3.12:

```
py -3.12 -m venv .venv
```

Allow activation in the current PowerShell session and activate the environment:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

After activation, the terminal should begin with (.venv).

6. Install Dependencies

Use python -m pip instead of direct pip to avoid PATH issues:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

7. Seed the Workspace

Seed default agents and workflow templates:

```
python scripts\seed_workspace.py
```

Expected output:

```
Seeded 5 agents and 3 workflows
```

If app module is not found

Run $env:PYTHONPATH="." and then run python scripts\seed_workspace.py again.

```
$env:PYTHONPATH="."
python scripts\seed_workspace.py
```

8. Set Environment Variables Before Starting the App

Set these variables in the same terminal where you will start FastAPI:

```
$env:Path += ";$env:LOCALAPPDATA\Programs\Ollama"
$env:OLLAMA_URL="http://127.0.0.1:11434"
$env:OLLAMA_MODEL="llama3.2:3b"
$env:OLLAMA_TIMEOUT_SECONDS="240"
```

These variables ensure the app uses the correct local LLM endpoint and has enough time for multi-agent execution.

9. Start the FastAPI Server

Start the app without --reload for stable Windows environment behavior:

```
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Open the web UI:

```
http://127.0.0.1:8000
```

10. Verify LLM Connection from the App

Open these URLs in the browser:

```
http://127.0.0.1:8000/api/llm/status
http://127.0.0.1:8000/api/llm/test
```

Expected status response:

```
{
  "available": true,
  "mode": "ollama",
  "url": "http://127.0.0.1:11434",
  "model": "llama3.2:3b",
  "model_installed": true
}
```

11. Run a Workflow

In the UI, open Workflow Builder and run this first short test prompt:

```
Summarize this issue and create 3 action items: EMR job failed due to memory issue but completed after memory increase.
```

Expected behavior:

- The orchestrator receives the task.
- Specialist agents execute their assigned tools.
- The local LLM generates responses using tool outputs.
- Action items appear in the result table.
- Logs, messages, metrics, and token usage update in the UI.

12. Configure WhatsApp Channel

Open Messaging Channels and fill WhatsApp Connector Setup. Minimum required values for sending messages are Access Token, Phone Number ID, Graph API Version, and Test Recipient Mobile.

Field	Value / Notes
Access Token	Meta WhatsApp Cloud API token
Phone Number ID	Phone number ID from Meta API Setup
WABA / Business Account ID	WhatsApp Business Account ID
Graph API Version	v25.0 or version shown by Meta
Test Recipient Mobile	Use country code, no plus sign; example: 919876543210
Verify Token	Any custom string for webhook verification
Default Workflow ID	1
Auto-send agent reply	Keep checked to send final agent response to WhatsApp

13. Final One-Shot Setup Command

Use this from the project root after Ollama is installed:

```
$env:Path += ";$env:LOCALAPPDATA\Programs\Ollama"

ollama pull llama3.2:3b

py -3.12 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
python -m pip install -r requirements.txt

python scripts\seed_workspace.py

$env:OLLAMA_URL="http://127.0.0.1:11434"
$env:OLLAMA_MODEL="llama3.2:3b"
$env:OLLAMA_TIMEOUT_SECONDS="240"

python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Common Errors and Fixes

Issue	Reason	Fix
ollama is not recognized	Ollama PATH is not visible to current terminal	Add $env:LOCALAPPDATA\Programs\Ollama to PATH or reopen VS Code
503 Service Unavailable on workflow	App cannot reach LLM or LLM timed out	Check /api/llm/status, set OLLAMA_URL, increase OLLAMA_TIMEOUT_SECONDS
PowerShell blocks venv activation	Execution policy restriction	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
pip is not recognized	pip is not in PATH	Use python -m pip install -r requirements.txt
No module named app	Script cannot locate project root	Set $env:PYTHONPATH="." or use fixed seed script
First workflow is slow	Model is loading into memory	Warm up with ollama run llama3.2:3b "hello"

Recommended final test

Before submission, run /api/llm/status, /api/llm/test, one workflow from the UI, and one WhatsApp test message. This confirms the LLM runtime, workflow engine, persistence, UI, and channel configuration are working.

Official References

- Ollama Windows download: https://ollama.com/download/windows
- Ollama Windows documentation: https://docs.ollama.com/windows
- Ollama local API base URL: https://docs.ollama.com/api/introduction
- Ollama local API authentication note: https://docs.ollama.com/api/authentication